Dev Tools

by Mo Bitar (@mo8it)

So you want to be a developer and you wonder what hard[1] skills you should learn besides programming? Then this course is for you!

This course will not teach you how to program. It will not teach you any algorithms or data structures. It is designed to teach you skills that almost every developer needs but are not taught in lectures or the like.

It all starts with the terminal.

You can try to run away from the terminal and look for a GUI (graphical user interface) for every task. But learning how to work in the terminal will unlock powerful capabilities as a developer and bring you to a next level of productivity. Not only will you be able to host and administrate servers (since they usually don't have a GUI). You will also be able to automate tasks and glue programs together like a wizard 🪄

When we talk about about the terminal for developers, we of course mean the Linux terminal 🐧

Windows also has a "terminal", but it is so useless that Microsoft itself started to offer a Linux system inside Windows[2]. MacOS being a Unix-like operating system offers a terminal a bit similar to that in Linux, but it is also useless in the server world.

After getting familiar with the Linux terminal, you will learn how to use one of the most important tools for developers: Git

Git is a version control system. You need to be familiar with its basics not only to collaborate with others, but also to manage your own software projects with a proper versioning and history.

Afterwards, you learn how to automate tasks using shell scripts and containers.

This course is a 5 days course[3] and should be read from the start to the end. If you skip some sections, you might not understand everything in later sections.

The course contains tasks to help you practice and experiment with what you have learned.

Are you excited? Let's start the journey!

---

1. Hard skills as the opposite of soft skills; not hard as in difficulty. ↩

2. See Windows subsystem for Linux which actually should be called “Linux subsystem for Windows”. ↩

3. The course started as a vacation course on the Johannes Gutenberg university in Mainz, Germany. ↩

Day 1

In this day, you will learn the basics of the Linux terminal.

But before we start with the basics, the first section elaborates on why you should learn Linux in the first place.

Why learn Linux?

It is important to answer this question before digging deeper into the course. Why would you want to learn Linux?

In this course, we are referring to Linux as an operating system (not only the kernel). Linux is the leading operating system when it comes to servers and computer clusters. Already since November 2017, all supercomputers on the top 500 supercomputers list are running Linux[1].

If you want to build anything on the internet (website, cloud services, etc.), you need a server and therefore you need Linux. If you want to do numerical computation on a high scale or anything related to the high performance computing field, you need Linux. The cluster Mogon 2 in Mainz (where this course is/was held) is no exception[2].

If you are not into servers and high performance computing, then Linux will still be interesting for you as a developer!

Many of development tools and programs run only on Linux. Linux provides you with awesome programs (especially CLI) that could make your life easier, not only as a developer, but also as a person that does more than internet browsing and document editing on the computer. Linux allows you to do exactly what you want efficiently. With the help of a shell script as an example (you will learn more about it later), you can automate a task that takes time and has to be done frequently.

On top of all of that, Linux is a set of free open source software. This means that you can use, read and study the code of your operating system and also send change requests to its developers. Everything is transparent! You can modify free software and redistribute it without any problems. While using Linux, you enjoy the freedoms of free software and control your computer instead of it being controlling you through closed source software that can not be studied. You know exactly what runs on your hardware!

Linux can also be used as a desktop! If you are interested in a free open source operating system that respects you as a user and your privacy and provides you with all the benefits mentioned above and more, then look no further! Example distributions are Fedora and Mint. Learning how to deal with the command line will make your experience smoother while using a Linux desktop. You will get a better understanding of the whole system and its tools after this course which will enable you to use a Linux desktop like a pro 😉

In the next chapter, we will actually start our learning journey with the terminal.

---

1. https://www.top500.org/statistics/details/osfam/1/ ↩

2. https://www.top500.org/system/178930/ ↩

Terminal basics

The terminal, the console, the command line. All three terms refer to the same thing.

Get access to a Linux system and open a terminal.

Warning ⚠️ : **Don't panic!**

Many find the terminal frightening. This is the cryptic thing you see hackers in films using! You might ask yourself, what am I doing here? I don't want to break my system or launch a terminator! 😳

Again, **don't panic!** What you see is just a prompt (usually on a dark background).

Now, how to interact with the terminal? You are probably not seeing a lot of buttons on your terminal. This is because you interact with it (mainly) through your keyboard. But what to type?

It is like a chat with your computer. But your computer is not into memes. It is a soldier waiting for your commands 💂

Echo

Let's enter our first command! Type `echo "Hello!"` and press enter. This is the output:

```
$ echo "Hello!"
Hello!
```

Congratulation! You did just say hi to your computer and it did even reply! What a charm! 🤠

Well, to be more precise, you gave a command to your computer to say "Hello!" and it did so... I know, I did just brake the romance between you and your computer. I apologize. 🙄

`echo` is a command that prints out what you give as an argument. It might not seem to be a useful command. Why would I want the computer to echo my words? The importance of `echo` will be clear mainly when writing scripts. More about this in later sections!

Navigation

Let's try more commands. Type `ls` and press enter. What do you see?

The output is going to be the files and directories in your current path. `ls` stands for *list*.

What if you want to take a look at the content of a different directory at a different path?

To examine this, let's first create a new directory. Enter the command `mkdir empty_house`. Well, you don't see anything? Did the command do something at all? To check, run `ls` again. Now, you should see your new directory `empty_house` listed too! `mkdir` stands for *make directory*. `empty_house` is just a name for our new directory. You could have used `mkdir Images` for example to create a directory called `Images`.

Is `empty_house` really empty? Let's check.

Enter the command `cd empty_house`. Again, you don't see an output. But maybe you did notice that a part of your prompt changed from `~` to `empty_house`. This indicates that you are in your new directory. `cd` stands for *change directory*.

Now, enter the command `ls`. You should not be seeing anything because the directory is indeed empty, until now.

Create, rename, move

An empty house is a sad house. Let's give the house directory some friends to live within it. Enter the command `touch friend1.txt`. Now, enter `ls` again:

```
$ ls
friend1.txt
```

You should see the first friend of the empty directory. `touch` creates a file if it does not exist. It does also have another use case which is not relevant at this moment.

But wait, our directory which is named `empty_house` is not empty anymore! Let's fix that by renaming it.

Enter the command `cd ..` to go one directory back. The two dots `..` refer to the parent directory in Linux.

Now that you are back in `~`, enter `mv empty_house happy_house`. Now, enter `ls` again. You can see that `empty_house` does not exist anymore. It was renamed to `happy_house` (*since it has at least one friend now*). `mv` stand for *move*.

But moving is not the same as renaming, right?

Well, `mv` does move a file or directory to a new destination with the possibility to give it a new name on the destination. So we did *move* the directory `empty_house` to the same location, but we did give it a new name.

I know, it is tricky. Let's take a look at an example that does actually *move*. Enter the command `touch friend2.txt friend3.txt`. This will create two new files at the same time. This way, you don't have to type `touch friend2.txt` **and** `touch friend3.txt`.

Now, let's move one of our new text files. Enter `mv friend2.txt happy_house`. Enter `ls` to see that `friend2.txt` disappeared. Let's verify that it now lives in `happy_house`. You could use `mv happy_house` and then `ls` analogously to the strategy above. But it is faster to use `ls` directly with a path as an argument. Enter `ls happy_house`:

```
$ ls happy_house
friend1.txt friend2.txt
```

We verified that `friend2.txt` was moved. Let's move `friend3.txt`, too. Enter `mv friend3.txt happy_house/loud_friend.txt`. Take a look at the content of your directory now:

```
$ ls happy_house
friend1.txt friend2.txt loud_friend.txt
```

We did not only move `friend3.txt`. We did also give it a new name in the destination. Hopefully, you have now a better understanding of renaming and moving with `mv`.

Remove

What if our house is not really *happy* anymore since a loud friend did move in? Let's remove that loud friend!

Enter `cd happy_house` and then `rm loud_friend.txt`. You will not see any output, but let's see what has changed in the directory:

```
$ ls happy_house
friend1.txt friend2.txt
```

The loud friend is removed! `rm` stand for *remove*.

> **Warning** ⚠️ : `rm` deletes a file directly! The file is **not** moved to a trash! It is gone! You can't just restore it anymore! Think twice before using `rm`.

> **Note**: Although `rm` deletes the file from the file system, there is still some chance that the file content still exists on the disk. One could try to recover it with some tools, but it is hard and nothing is guaranteed. Nevertheless, if you want to delete something sensitive

for sure, then you have to use a tool that overwrites the file content before deleting it. Physically destroying the whole disk is also an option 🔥😂

Does `rm` also work with directories? Let's test it:

```
$ ls
friend1.txt friend2.txt

$ mkdir zombies

$ ls
friend1.txt friend2.txt zombies

$ rm zombies
rm: cannot remove 'zombies': Is a directory
```

So it does not work. To delete an empty directory, use `rmdir`:

```
$ rmdir zombies

$ ls
friend1.txt friend2.txt
```

`rmdir` stands for *remove directory*. But it does only work on empty directories! Let's verify this on none empty directories:

```
$ ls
friend1.txt friend2.txt

$ mkdir zombies

$ touch zombies/zombie1.txt

$ ls zombies
zombie1.txt

$ rmdir zombies
rmdir: failed to remove 'zombies': Directory not empty
```

What to do now? Turns out, `rm` is more powerful than we thought and can delete directories too, even if they are not empty! But you have to give it a *flag*.

What is a flag? It is an option that you specify after the command name that enables some functionality.

In our case, to remove a directory that is not empty, we need the `-r` flag of `rm`:

```
$ tree
.
├── friend1.txt
├── friend2.txt
└── zombies
    └── zombie1.txt

1 directory, 3 files

$ rm -r zombies

$ ls
friend1.txt friend2.txt
```

The directory `zombies` is removed!

You probably wonder what `tree` is. As you can see in the output above, it shows you the tree of your current directory 🌳 It goes recursively through all directories starting with the current path and shows their content.

Simple file editing

Now, we want to write some rules to avoid having loud people or zombies again.

Let's make sure that we are in our house directory first. To do so, enter `pwd`:

```
$ pwd
/home/USERNAME/happy_house
```

What you see is the path to your current directory (also called working directory). *USERNAME* here is the name of your user. `pwd` stands for *print working directory*.

We could write our rules in a simple text file. But how do we edit files in the terminal?

There are many ways to edit text files in the terminal. The simplest method is to use the editor `nano`.

Enter the command `nano rules.txt`. You will be presented with a "terminal window" showing a blinking cursor and some shortcuts at the bottom.

Let's write some rules, for example:

```
No zombies!
No loud housemates!
```

How to save and exit? Let's take a look at the shortcuts at the bottom. The shortcuts starting with the symbol `^` expect the `Ctrl` key. Those starting with `M-` expect the `Alt` key. To save, we use `^X` which means that we have to press `X` while holding `Ctrl`.

Now, it asks us:

```
Save modified buffer?
Y Yes
N No      ^C Cancel
```

A buffer is a file that is loaded in the memory (RAM). If you press `N`, then your edits are lost! In our case, we want to save the rules. Therefore, we press `Y`.

Now it shows us:

```
File Name to Write: rules.txt
```

We want to keep the name that we did specify while calling `nano`. Therefore, we just press `Enter`. Otherwise, you can change the name and save the file under another name.

Let's take a look at our rules:

```
$ ls
friend1.txt friend2.txt rules.txt

$ cat rules.txt
No zombies!
No loud housemates!
```

We used `cat` 😺 to only print the content of the file `rules.txt` without opening it with an editor. `cat` 🐈 stands for *concatenate*. This does not sound like printing file content!? This is because `cat` 😺 can be used to concatenate the content of multiple files. But this is not relevant for now.

Manuals

After a while, the house might have many rules, more than the two that we just entered. It would be useful to be able to count how many rules we have. To do so, we can use the command `wc` which actually stands for *word count* (not 🆎), but it is able to count more than words:

```
$ wc rules.txt
2 5 32 rules.txt
```

We see three numbers. The first one is the number of lines, the second is the number of words and the last is the number of bytes. How do I know that?

I do not memorize everything! I had to look up the meaning of the three numbers.

Where do you look up such things?

You could use the internet, but (almost) every Linux command comes with a manual. To access the manual of a command, we use the command `man` followed by the name of the command that we want to look up. You probably already guessed it, `man` stands for *manual*.

Let's look up the command `wc`:

```
$ man wc
NAME
    wc - print newline, word, and byte counts for each file

SYNOPSIS
    wc [OPTION]... [FILE]...
    wc [OPTION]... --files0-from=F

DESCRIPTION
    Print  newline,  word, and byte counts for each FILE, and a total line
    if more than one FILE is specified.
    A word is a non-zero-length sequence of printable characters
    delimited by white space.

    (…)

    -m, --chars
        print the character counts

    -l, --lines
        print the newline counts

    (…)

    -w, --words
        print the word counts

    (…)
```

`(…)` indicates that parts of the output were kept out to only show output relevant to our context.

The structure of the output is a standard for command manuals (also called *man pages*).

The first section of the output is *NAME* which shows the name of the command with a brief description of what it does.

The second section is *SYNOPSIS* which shows different ways of using the command. The brackets indicate that a part is optional. The three dots `...` (not `(…)`) indicate that a part can be specified more than one time. This means that `[OPTION]...` tells us that the command accepts zero to many options.

The options are presented in the section *DESCRIPTION* which also explains the command but with more details.

Let's pick one option: `-l, --lines`. The first part `-l` shows the short version of the option (not always given). The second part `--lines` shows the long version which is more verbose. The description below tells us that the option prints the newline counts. We can try it out, but first, you might already be in the panic mode because you might not find out how to exit this manual.

HOW CAN I EXIT THIS MANUAL? DO I HAVE TO EXIT THE TERMINAL AND OPEN A NEW ONE? 😱

Don't panic! (*Wait until you open Vim*)

You can exit the manual by pressing the key `q`. You are free now 🕊️

Now, let's try the option `-l` that we got from the manual:

```
$ wc -l rules.txt
2 rules.txt

$ wc --lines rules.txt
2 rules.txt
```

You can see that the short `-l` and long `--lines` versions are equivalent. They lead to returning the number of lines.

Let's try using more than one option:

```
$ wc -l -w rules.txt
2 5 rules.txt
```

We see that we get the number of lines and then the number of words.

We learned how to access and read the manual, but how do we navigate the manual?

If you open a manual with `man`, you can scroll up and down using the arrow keys. You can search by pressing `/`, then enter what you are search for, `lines` for example, and then press `Enter`.

If you get more than one match for your search pattern, you can jump to the *next* match with the key `n` and to the previous one with `N` (`Shift + n`).

You can learn more about how to navigate manuals (and other so called pagers) by pressing `h` in a manual. But since a lot is derived from the editor Vi/Vim, we will learn more about it later when we learn about this editor.

Fun fact: You can read the manual of the manual with `man man` 😀

Help!

If you want to read a smaller version of the manual, you can run `COMMAND --help`. Almost every command supports the `--help` option and its small version `-h`. You will then get a quick reference.

Try `wc --help`!

Commands that don't have a man page usually have detailed help messages as an alternative.

Paths

Let's build one more house!

```
$ cd ~
$ mkdir new_house

$ cd new_house
```

We did not use any new commands, but what is this symbol `~`? We did already mention it and you might have noticed that it was shown in the prompt after opening the terminal. The so called *tilde* `~` stands for your *home directory* (home for short) in Linux 🏠 Your home as a user has the path `/home/USERNAME` and is the directory where you should place your own files.

By entering `cd ~`, we did make sure that we are in our "home" before creating a new directory. Entering only `cd` does have the same effect as `cd ~`.

So we did create a new house directory and move into it. Let's say we want to copy the rules from the first house to the new one. To do so, we can use the command `cp` that stand for *copy*:

```
$ pwd
/home/USERNAME/new_house

$ cp ~/happy_house/rules.txt .

$ ls
rules.txt

$ cat rules.txt
No zombies!
No loud housemates!
```

We copied the rules, but did you notice the dot at the end of the command `cp`? What does it mean?

We learned that the two dots `..` refer to the parent directory (one directory back). One dot `.` refers to the current directory.

Here are some **equivalent** commands that might help you understand paths in Linux:

1. `cp /home/USERNAME/happy_house/rules.txt /home/USERNAME/new_house/rules.txt`
2. `cp ~/happy_house/rules.txt ~/new_house/rules.txt`
3. `cp ~/happy_house/rules.txt ~/new_house`
4. `cp ~/happy_house/rules.txt .`
5. `cp ../happy_house/rules.txt .`

All the commands above do the same thing. The difference is the way they specify the source and destination path.

The **1. command** is the most verbose one. We specify the full path of the source and destination.

In the **2. command**, we use `~` which is a shortcut to `/home/USERNAME`.

In the **3. command**, we remove the file name from the destination path. If a file name is not specified in the destination path, then the file name from the source is used.

In the **4. command**, we use the dot `.` as a shortcut to the current directory since we are currently in the directory `new_house`.

In the **5. command**, we use the two dots `..` instead of `~` which is also possible since the parent directory is `~` in this case. The two dots are useful when we are operating in directories that are nested deeper in the home directory.

You might think: `.` is the path to the current directory. `..` is the path of the parent directory. Is `...` the path of the parent directory of the parent directory? 🤯 Let's try it out:

```
$ ls .
rules.txt

$ ls ..
happy_house new_house (...)

$ ls ...
ls: cannot access '...': No such file or directory
```

So this does **not** work. Any number of dots greater that 2 doesn't work, except if we use separators. To access the parent directory of the parent directory, we can use `../..`:

```
$ ls ../..
USERNAME (...)
```

You will probably only see your user name as output, but if the system you are using has more than one user, then names of the other users would be in the output, too. Normally, every user has an own home directory under `/home`.

The usage of `../..` or even more dots like `../../..` is not recommended since you would have to go multiple directories up in your mind and this does not work well! I would recommend using the full path for paths not in the current `.` or in the parent directory `..`.

What we have just learned about paths does not only apply to `cp`, but also to `ls`, `mkdir`, `mv` and all commands that deal with paths.

One thing has to be mentioned about `cp`: To copy directories instead of just files, use the option `-r` to copy *recursively* like with `rm`.

Terminal shortcuts

For typing a long path, you can use **autocompletion**. Go back to the home directory using `cd`. Let's say that we want to read the rules of the directory `happy_house`. Type `cat hap` without hitting enter yet. Now, press `Tab` and see how the path is autocompleted to `cat happy_house/` ✨ Now, type `ru` and hit `Tab` again. Then you have `cat happy_house/rules.txt`. Much faster, right? 🏎️

If you have another directory in your home that is called something like `happy_directory`, then the autocompletion completes only to `cat happy_`. Pressing `Tab` again shows you the possible completion options. Type one or more characters to make an autocompletion possible and then hit `Tab` again to have the directory name autocompleted.

Often, commands are not long, but you can use autocompletion to complete commands too. If you type `ech` and then press `Tab`, you get the autocompletion `echo` with the space at the end to enter options or arguments. In this case, we did only save entering one character and a space, so `echo` is not the best opportunity for autocompletion. But this is only a demonstration. Some command names are longer.

You might think that you don't need autocompletion at all. But you should use it. Not only for the autocompletion itself, but for verification, too!

When you type `cat happy_house/ru` and then press `Tab`, you don't get any autocompletion although you would expect it. But did you notice the missing `p` in `happy`? 🧐

This is meant by verification. If you don't get an autocompletion although you think that you should, then check what you have typed so far. It is much easier to correct things this way than having to correct them after trying to run the command.

But what if you did run a command with a mistake and you have to correct it? Or what if you just want to run the same command again with small modifications? Do you have to type the whole command again?

Fortunately, no! You can use the up ⬆️ and down ⬇️ arrow keys to navigate through your commands history. Try it out!

If you start typing a command and you notice that you missed something in the middle of the command, you might try to use the mouse and click. No clicks will help you! Instead, use the left ⬅️ and right ➡️ arrow keys.

If you want to go to the beginning of the command, press `Ctrl + a`. Then you can go back to the end with `Ctrl + e`.

If you want to close a session quickly, press `Ctrl + d`. For now, this will close your terminal. Later when we use SSH to access other Linux machines, `Ctrl + d` will close the connection and bring you back to the prompt of your machine.

Let's write a very small program in Python. Enter `nano nonstop.py` and type in the following small snippet:

```
from time import sleep

while True:
    print("You can't stop me, can you?")
    sleep(1)
```

Now, exit the file while saving the buffer. If you don't know about Python, don't worry. You don't need any real programming skills for this course. All that you have to know is that this program

runs forever and prints “You can’t stop me, can you?” every second.

You shouldn’t write endless loops. This is only a demonstration for a program that is running and you would like to stop.

Enter `python3 nonstop.py` to run the code.

Annoying, right? To stop a running program, press `Ctrl + c`. You *have to* remember this shortcut!

If you are typing a command and want to start over, you can use `Ctrl + c` instead of pressing `Backspace` repeatedly.

But wait, `Ctrl + c` is (normally) used for copying, right?

Not in a terminal! If you want to copy text in a terminal, then select it with the mouse and the press `Ctrl + Shift + C`.

To paste text into the terminal, press `Ctrl + Shift + v`.

Why is copying and pasting so inconvenient? It has historical reasons. You have to take it as it is 😟

It might be overwhelming to memorize all commands, shortcuts and options. If a command is an abbreviation, then knowing what it stands for is very helpful. If you want to use an option but you are not sure which flag it was, then use `--help` or read the manual with `man`! Otherwise, you can look up things in this course or using a search engine 😉

Packages

So far, we did only use commands that are common in every Linux system.

Now, we want to start exploring additional commands that might not be preinstalled on your system.

Therefore, we need to learn how to install so called *packages* to get access to additional programs.

Package managers

In Linux, you don't install programs by using a search engine to find a website, then visit the website and download an installer, then run the installer and click "Next" and "Agree" repeatedly without even reading what you are agreeing to 😫

This method is too inconvinient and insecure. You have to trust the website that you visit. You have to hope that no malware is delivered while you are downloading the installer from the website. You can get the malware either from the website itself or through a man-in-the-middle attack. Then the installer might do whatever it wants. You just click "Next" and suddenly your system shows you (extra) ads or your browser search preferences are overwritten by Yahoo. Surprise! 🎉

In Linux, you (mainly) install software through the package manager that is shipped with your distribution. The software is then downloaded from a repository which is a collection of packages that is managed by the distribution. The distribution managers make sure that no malware is added to their repositories.

A package manager takes care of verifying the downloaded packages before installation. This is done using some cryptographic methods and prevents man-in-the-middle attacks or installation of packages that were corrupted during the download process.

The package manager does also take care of installing any other packages that you did not specify but are required by the package that you want to install. These packages are called *dependencies*.

Since this course uses a Fedora system for its demonstrations, we will use Fedora's package manager `dnf`.

If you are using another Linux distribution, you might need to replace the commands of `dnf` with `apt` for Debian based distributions for example.

It is important to understand the functionality of package managers. Then you can transfer this knowledge to other package managers if required.

This applies to some other aspects in Linux too, since ***the*** *Linux operating system* does not exist. There are distributions that bundle a set of software and differ in small ways.

To install a package, you need administrative privileges. Therefore, we need some `sudo` powers. `sudo` will be explained in the next section.

Let's install our first package! To do so, enter `sudo dnf install cmatrix`. You will be asked for the password of your user. Type the password and then press `Enter`. For security reasons, you will not be able to see you password while entering it. So don't wonder why nothing happens while typing the password. Just type it and then press `Enter`.

The package manager might need some time to synchronize some data, then it will show you a summary of what it will do and ask you for confirmation. Type `y` for *yes* and then press `Enter` for the installation to start.

After the installation is done, you can enter `cmatrix`. Congratulations, you are a hacker now! 💻🤓

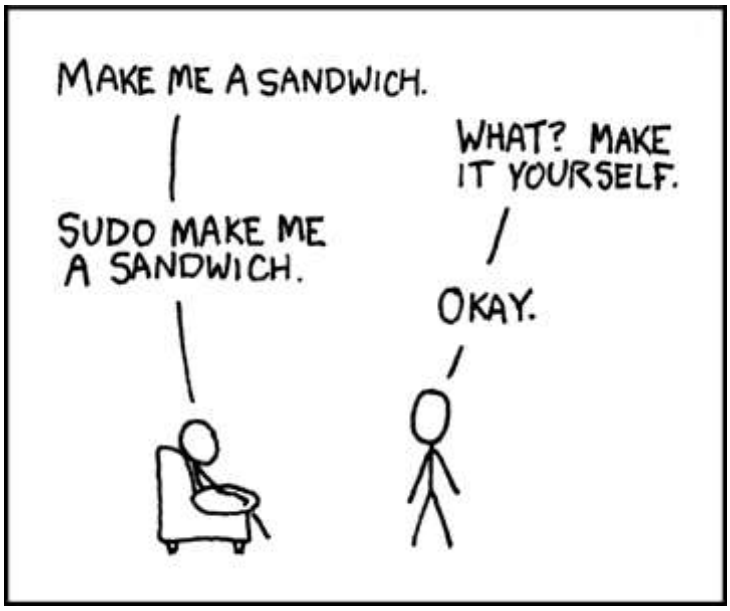
To exit the matrix, press `q`.

What if you don't like the matrix and want to remove it? You can uninstall packages using `sudo dnf remove PACKAGENAME`. In this case: `sudo dnf remove cmatrix`. You have to confirm again with `y`.

Why do we speak about *packages* instead of programs when installing software on Linux? Because packages can contain more than one binary (the actual program) and extra files. Take a look at the files that are installed with `cmatrix` on Fedora for example.

> Warning ⚠️: While installing or uninstalling packages, it is important to take a look at the summary before confirming the action with `y`. There is a reason why a confirmation is required. Sometimes, some packages depend on packages with older versions than the versions that exist on your machine. If you want to install those packages anyway, other packages that depend on the newer versions could break!
>
> Just don't blindly press `y`. If the package manager gives you a warning before doing something, enter this warning in a search engine and read about what it means before continuing!

Sudo



Let's try to install another package. But this time without `sudo` at the beginning of the command.

```
$ dnf install cowsay
Error: This command has to be run with superuser privileges (under the root user
on most systems).
```

You can see that you get an error resulted by a lack of privileges for running this command.

Any action that might modify the system needs administrative privileges. Installing a package (system-wide) is one of these actions.

Sometimes, an action might not modify the system, but a user might be restricted through some permissions to not be able to access a file for example. In this case, you would also need to use `sudo`.

Linux has a superuser, one user that exists on every Linux system and is able to do anything! This user is the `root` user. For security reasons (and also to not do something destructive by a mistake), this user is often locked. `sudo` allows you as a non root user to run commands as a `root` user.

You are not always allowed to use `sudo`. If you don't own the machine you are using and just share it with others, then there is a high chance that only administrators of the machine have `sudo` access.

If you don't have access to `sudo` and try to use it, then you get an output like this:



Warning ⚠️ : Use `sudo` with great caution! Do not run a command from the internet that you don't understand! Especially if it needs to be run with `sudo` ! RED FLAG! 🔴

Look up a new command before you run it.

Even commands that you know like `rm` can destroy your system if you use them with `sudo` without knowing what they are doing.

You might find a "meme" in the internet that tells you to run something like `sudo rm -rf /` . This command would delete EVERYTHING on your machine. It is like deleting the `C` and all other drives at the same time on Windows.

Linux assumes that you know what you are doing when you use `sudo` . *With great power comes great responsibility!*

Looking for a package

If you don't exactly know the name of a package you are looking for, then you can use **`dnf search PATTERN`** . DNF will then return packages that match this pattern in their name or description.

For example, if you search for Julia (a programming language) you get the following results:

```
$ dnf search julia
============================== Name Exactly Matched: julia
==============================
julia.i686 : High-level, high-performance dynamic language for technical computing
julia.x86_64 : High-level, high-performance dynamic language for technical
computing
============================== Name & Summary Matched: julia
==============================
cantor-julia.i686 : julia backend for cantor
cantor-julia.x86_64 : julia backend for cantor
julia-common.noarch : Julia architecture-independent files
julia-devel.i686 : Julia development, debugging and testing files
julia-devel.x86_64 : Julia development, debugging and testing files
julia-doc.noarch : Julia documentation and code examples
perl-DateTime-Calendar-Julian.noarch : Julian Calendar support for DateTime.pm
vim-syntastic-julia.noarch : A syntax checker for julia programming language
================================ Summary Matched: julia
================================
jday.i686 : A simple command to convert calendar dates to julian dates
jday.x86_64 : A simple command to convert calendar dates to julian dates
perl-Date-JD.noarch : Conversion between flavors of Julian Date
python3-jdcal.noarch : Julian dates from proleptic Gregorian and Julian calendars
```

If you know the name of the program but you don't know the name of the package that contains this program, use **`dnf provides PROGRAMNAME`**.

If you want to install the program `trash` (no joke 🗑) that provides you with the functionality of a system trash (instead of completely deleting), then you can run this:

```
$ dnf provides trash
trash-cli-0.21.10.24-2.fc36.noarch : Command line interface to the freedesktop.org
trashcan
Repo        : fedora
Matched from:
Filename    : /usr/bin/trash

trash-cli-0.22.4.16-1.fc36.noarch : Command line interface to the freedesktop.org
trashcan
Repo        : updates
Matched from:
Filename    : /usr/bin/trash
```

You can see that the name of the package `trash-cli` is not the same as the name of the program/binary `trash`. Using `provides` makes your life easier while looking for the package to install.

If you did find a package but you want to get more information about it, you can use **`dnf info PACKAGENAME`**:

```
$ dnf info julia
Available Packages
Name         : julia
Version      : 1.7.3
Release      : 1.fc36
Architecture : i686
Size         : 45 M
Source       : julia-1.7.3-1.fc36.src.rpm
Repository   : updates
Summary      : High-level, high-performance dynamic language for technical
computing
URL          : http://julialang.org/
License      : MIT and LGPLv2+ and GPLv2+
Description  : Julia is a high-level, high-performance dynamic programming
language
             : for technical computing, with syntax that is familiar to users of
             : other technical computing environments. It provides a sophisticated
             : compiler, distributed parallel execution, numerical accuracy, and
an
             : extensive mathematical function library. The library, largely
written
             : in Julia itself, also integrates mature, best-of-breed C and
Fortran
             : libraries for linear algebra, random number generation, signal
processing,
             : and string processing.
             :
             : This package only contains the essential parts of the Julia
environment:
             : the julia executable and the standard library.

Name         : julia
Version      : 1.7.3
Release      : 1.fc36
Architecture : x86_64
(…)
```

You get two results for Julia that seem to be identical. The difference is the *architecture*. Normal computers usually have the `x86_64` architecture, so you should look for it. But it is nice to know that other architectures are also supported.

To run updates on your system, use **`dnf upgrade`**.

For security reasons, it is important to run updates frequently! A package manager updates the whole system and its programs. This means that you don't have to update your packages separately.

If you have a problem with a package after an update, then you can try to use an older version. To do so, refer to the documentation of **`dnf downgrade`**.

Tasks

Task: Collective-Score

This course uses collective-score for interactive tasks. I spent many days programming it for you 😇

As an attendee of the course, you will find the binary `cs` on the Linux system provided to you. Check it by running `cs --help`.

You should use it to validate that you have done a task correctly. But first, you need to register yourself to be able to communicate with the Collective-Score server and see your progress on the projector.

Run the following command **after replacing `USERNAME` with your username** (you can choose whatever username you want to be displayed to others):

```
cs user register USERNAME
```

If you get any errors, ask the tutor before proceeding with the tasks!

Now, run the following command:

```
echo "Test" | cs task collective-score-intro
```

The check that `cs` does should fail. This is a demonstration of how checking a task fails and how you (hopefully) get a hint.

In general, you will do a task and then run `cs task TASKCODE` to validate your solution. `TASKCODE` is a code unique to every task/subtask. For this demo task, the task code is `collective-score-intro`.

🟢 Now, for this demo task to pass the check, run the following command:

```
echo "OK" | cs task collective-score-intro
```

Congratulations, you have done your first task 🎉

To see your progress at any time, run the command **`cs progress show`**.

Task: Building houses 🏠

In this task, you will build a house with different rooms using (nested) directories.

Start with an empty directory as the house and add directories to it as rooms.

Rooms (represented by directories) contain objects as text files (ending with `.txt`). The text files for the objects should be empty for now.

The house should have the following 5 rooms:

- `bedroom` containing `bed.txt`
- `bathroom` containing `toilet.txt`
- `living_room` containing `couch.txt`
- `kitchen` containing `fridge.txt`
- `dining_room` containing `chair.txt`

Place Max in the living room by creating a file `Max.txt` in the living room containing his hobby `beatboxing` as text in his file.

Example:

```
$ cat living_room/Max.txt
He is watching TV.
His hobby is beatboxing.
```

Run the command `tree` to see the structure of the house.

🟢 Run `cs task initial-house` in the house directory.

Zombies are back! They did get in through *windows*... (You got it?)

Place a zombie in the living room having `Brainzzz` in his file `zombie.txt`.

Max runs out of the living room! Move him to the bedroom.

🟢 Run `cs task zombie` in the house directory.

Now, destroy the whole living room with the zombie in it.

🟢 Run `cs task zombie-nuked` in the house directory.

Hints

- If you get lost, use `pwd`.
- If you are looking for an option but you can not remember it, use `--help`.

Task: Password 🔑

⚠️ Launch the fish shell by running the command `fish` and stay in it for all following tasks. If you close the terminal and launch it later again, you have to run `fish` again. We will learn more about the fish shell later.

Use the command `passwd` to reset the password of your user. It is important to have a secure password!

🟢 Run `cs task passwd`.

Task: System updates

Find out how to update your system with `dnf` and run the updates.

Before you confirm, make sure that you read what packages are updated. Make a screenshot of the changes.

It is possible that you don't find any updates. In this case, you can try it again on another day!

🟢 Run `cs task system-update`.

Task: Package installation and usage

Install the package `cowsay` and find out how to use it!

Spoiler: You will have to deal with some cows 🐄

Find out how to give a fancy tongue and shinny eyes.

🟢 Run `cs task cowsay`.

Now, install the package `lolcat`.

Generate some wisdom with `cowsay (…)`. Now, run `cowsay (…) | lolcat`. What did change? Can you guess how this internally works?

Now, read the help of `lolcat` and find out how to add some randomness.

- Run `cs task lolcat`.

Day 2

In this day, we will learn more about the terminal and how to glue commands together to benefit from their modularity.

But we will start by making our terminal more comfortable! Let’s make an upgrade away from the terminal of the 90s ✨

Terminal upgrade

Fish

When you run something in the terminal, then you are interacting with a so called *shell*.

The default shell on almost all Linux systems is `bash`. Therefore, you should be familiar with it.

But if you want a modern terminal experience instead of that of the 90s, then you should use the Fish shell 🐠 The **f**riendly **i**nteractive **sh**ell".

Bash offers basic (auto)completion, but Fish takes it to the next level!

Type `ls ~/` (without hitting `Enter` yet) and press `Tab` twice in Bash. Bash will just show you all possible completion options. But Fish will let you cycle through these options with `Tab` and `Shift + Tab` to choose one! This doesn't only work with paths, but also for commands and even command options!

In Bash, you you can cycle through your command history using the up and down arrow keys. But in Fish, you can also start a command and *then* cycle through your history that has the same **prefix** with the up and down arrow keys!

It will also automatically give you suggestions to what you type based on history and completion! These autosuggestions are showed as dimmed text to the right of your input. To use that suggestion, you can just press the right arrow key (or `Ctrl + e`)!

Fish also supports true color synatx highlighting 🌈

Colors are not only fancy, but they can also be very helpful! If you start typing a command with a program that does not exit (like `echoo` instead of `echo` for example), Fish will highlight that program in red! Otherwise, the program is highlighted in blue! This gives you rapid feedback while typing 🤩

You can also configure Fish in the file `~/.config/fish/config.fish`. Here is any example

```
if status is-interactive
    # Disable the greeting message.
    set -g fish_greeting

    # Abbreviations

    # Prevent overriding files/directories by accident when using `cp` and `mv`.
    abbr -ag cp "cp -i"
    abbr -ag mv "mv -i"
    # Use `trash` by default instead of `rm`.
    abbr -ag rm "trash"
    # Use another shortcut for when you really want to delete instead of moving
into a trash.
    abbr -ag rmr "rm -r"
    # Set some default options.
    abbr -ag rsync "rsync -Lahz"

    # Alias
    # Use `bat` instead of `cat` for colored output!
    alias cat "bat"

    # Functions
    function demo
        echo "Hallo from the demo function!"
        echo "Arguments taken: $argv"
        echo "First argument: $argv[1]"
    end
    # Well will learn more about arguments later.
end
```

Use **abbreviations** when you want your input to be replaced while typing with the ability to modify the replacement. **Aliases** should only be used if you are sure that the two commands are equivalent for you.

Zellij

Now that we have a fancy shell, what can be even better than a fancy terminal?

Multiple fancy terminals!

Let me introduce you to Zellij, a modern terminal multiplexer written in Rust 🦀

It offers panes, tabs and even floating panes!

Start Zellij by running `zellij` and take a look at the shortcuts at the bottom. It is pretty user friendly!

Press `Ctrl + p` to go into the *pane mode* where you can select one of the shortcuts that newly appeared at the bottom. Press `n` in the *pane mode* to create a **new pane**!

You can change the focused pane by clicking on another pane. But there are also keyboard shortcuts for that.

Normally, you can press `Ctrl + t` to go into the *tab mode* where you can press `n` to create a new tab. But if you are using the terminal in the browser provided during the course, then you might not be able to press `Ctrl + t` without opening a *browser* new tab.

Therefore, we have to change that keybinding to something that does not conflict with the browser. To do so, create the directory `~/.config/zellij` and place the file `config.kdl` in it with the following content:

```
keybinds {
    normal {
        bind "Ctrl b" { SwitchToMode "tab"; }
    }
}
```

This configuration uses the shortcut `Ctrl + b` instead of `Ctrl + t` to enter the *tab mode*.

Now, use that new shortcut followed by `n` to create a new tab. The easiest way to change the focused tab is by clicking on the tab label at the top. But there are also keyboard shortcuts for that.

Try detaching the session by pressing `Ctrl + o` followed by `d`. This will lead to exiting Zellij, but your session (open panes and tabs) is not gone!

To attach back to that session, run `zellij attach` or `zellij a` for short. Fancy, right? You can just detach a session you working in, do something else and attach back to continue from where you have left it 😀

There is still something annoying: Everytime you start zellij, you have to start fish afterwards. Wouldn't it be awesome to have Fish as the default shell in Zellij?

Let's make it the default one! Add the following line to the beginning of the configuration file `~/.config/zellij/config.kdl`:

```
default_shell "/usr/bin/fish"
```

You can check that this is the path of the program `fish` on your system by running `which fish`.

We are not done with upgrading our terminal workspace. We will introduce more modern tools later. But this should be enough for now 🙂

Shell glue

The shell has the power to glue commands together to make the impossible possible! Let's use some gluing magic!

Piping

We will start with pipes. In the last task of the last day, you did see the usage of the symbol `|` between two separate commands.

By entering `cowsay "Hello" | lolcat`, the output of the first command `cowsay` is sent to the second command `lolcat`.

`lolcat` takes the input, colors it and outputs it again!

Many Linux commands support handling input of another command.

You might have seen in the manual of `wc` in day 1 that the file as an argument is only optional. How could you use `wc` without arguments?

You might have guessed it now, make some `wc` pipes. OK, I admit that the naming is not the best 🆎😂

Let's get some data to work with. To do so, we will use the command `curl` which fetches content from the internet.

Let's count the number of lines of the HTML file of the homepage of this book:

```
$ curl -s https://dev-tools.mo8it.com | wc -l
254
```

The option `-s` tells `curl` to be silent and not show progress information.

You can see that `wc` did count the number of lines. We did just combine two completely different tools using a pipe!

How about counting the number of lines that contain the word "Linux" on the homepage? To do so, we will add a new pipe inbetween!

`grep` is a command that searches for matches of a specified pattern. Each line with a match is printed in a new line.

To demonstrate `grep`, here is one usage example:

```
$ curl --help | grep "silent"
 -s, --silent    Silent mode
```

We did just filter the output of the help message of a command. This is one way to search quickly for command options!

Back to the main example:

```
$ curl -s https://dev-tools.mo8it.com | grep "Linux" | wc -l
6
```

You can see that you can use multiple pipes. This allows for almost infinite combinations!

Being able to combine commands is the reason why many commands are simple. They do one thing and do it well! To do more, combine them!

This is much more flexible and powerful than a program that tries to do many things at once.

Input, output

Before going any further, we need to understand an important concept in Linux.

A command accepts input and generates two types of output. The input is called *standard input* (**stdin**). The output is split to *standard output* (**stdout**) and *standard error* (**stderr**). They actually have numbers that will be relevant later:

- 0: stdin
- 1: stdout
- 2: stderr

Normal output is sent to the standard output. Errors (and sometimes output that is not important) are sent to the standard error.

Redirections

You can redirect the standard output or the standard error of a command to a file!

If you just run `curl -s https://dev-tools.mo8it.com`, you will see the HTML file printed to the terminal. Let's redirect the output to an HTML file on the disk:

```
$ curl -s https://dev-tools.mo8it.com >dev-tools.html
```

Now, view the content of the new file `dev-tools.html`. You will see the same output from the terminal without redirection.

Now, try this command:

```
$ curl https://non-existent-site.mo8it.com >test.html
(…)
curl: (60) SSL certificate problem: self-signed certificate
More details here: https://curl.se/docs/sslcerts.html

curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the web page mentioned above.

$ cat test.html
```

You will see that the file is empty since `curl` did not find a page to show as normal output. The error message was displayed using `stderr`.

If you are using this command in a script, then it might be wise to redirect the error to a log file:

```
$ curl https://non-existent-site.mo8it.com 2>curl.log

$ cat curl.log
curl: (60) SSL certificate problem: self-signed certificate
(…)
```

You can see that the error is not shown anymore after running the `curl` command. It was redirected to the file `curl.log`.

Did you notice the number 2 before the redirection symbol `2>`?

The last section did mention that the number of the standard error is 2. Therefore, 2 has to be specified to redirect it.

If you don't specify a number, then it is equivalent to 1 which stands for the standard output. This means that `>` is equivalent to `1>`.

What if a command produces output using stdout *and* stderr? Take a look at the following example:

```
$ touch some_file.txt

$ # Produces output to stdout and stderr.
$ ls some_file.txt does_not_exist.txt
ls: cannot access 'does_not_exist.txt': No such file or directory
some_file.txt

$ # Redirect only stdout to a file. stderr is shown.
$ ls some_file.txt does_not_exist.txt >stdout.txt
ls: cannot access 'does_not_exist.txt': No such file or directory

$ cat stdout.txt
some_file.txt

$ # Redirect only stderr to a file. stdout is shown.
$ ls some_file.txt does_not_exist.txt 2>stderr.txt
some_file.txt

$ cat stderr.txt
ls: cannot access 'does_not_exist.txt': No such file or directory

$ # Redirect both stdout and stderr to different files.
$ ls some_file.txt does_not_exist.txt >stdout.txt 2>stderr.txt

$ cat stdout.txt
some_file.txt

$ cat stderr.txt
ls: cannot access 'does_not_exist.txt': No such file or directory

$ # Redirect stdout and stderr to the same file with `&>`.
$ ls some_file.txt does_not_exist.txt &>mixed.txt

$ cat mixed.txt
ls: cannot access 'does_not_exist.txt': No such file or directory
some_file.txt
```

Appending

So far, we did redirect output using the operators `>`, `1>`, `2>` and `&>`. But these operators overwrite the files they are redirected to if they already exist. If you want to append to a file instead, use the operators above but with double `>`, for example `&>>`.

Discarding

There is a special file that you see some command redirect to: `/dev/null`.

This file is like a black hole. Everything redirected to that file is discarded.

For example, if you don't care about the errors that some command throughs, then you can redirected its stderr to `/dev/null` using `2>/dev/null`

More details

We discussed the most important cases for redirections. But there are some less important details like the following:

- `command &>filename` is equivalent to `command >filename 2>&1`, but not to `command 2>&1 >filename` because the order matters.
- `0<filename command` or `command 0<filename` can be used to redirect a file to the stdin of a command.
- A pipe `command1 | command2` redirects only the stdout of `command1` to the stdin of `command2`. But if you want to redirect stderr too, then you have to use `command1 2>&1 | command2`. To only redirect stderr, you have to use `command 2>&1 >/dev/null | command2` (not `command >/dev/null 2>&1 | command2` since the order matters).

You might have noticed how it can get complicated. Therefore, refer to the "Redirections" section in the Bash reference manual for more details.

Yes, the reference is about Bash, but Fish has the same behavior here.

Shell tricks

We will learn about two shell features that can save you a lot of typing.

Expansion

`command PREFIX{aaa,bbb}POSTFIX` is expanded to `command PREFIXaaaPOSTFIX PREFIXbbbPOSTFIX`. It also works with more than two arguments inside the curly brackets `{}`.

This is especially useful when dealing with paths. Here are some examples:

- `mkdir -p dir/sub{1,2,3}` ➡️ `mkdir -p dir/sub1 dir/sub2 dir/sub3`
- `touch dir/sub1/file{1,2}.txt` ➡️ `touch dir/sub1/file1.txt dir/sub1/file2.txt`
- `cp dir/sub1/file1.txt{,.bak}` ➡️ `cp dir/sub1/file1.txt dir/sub1/file1.txt.bak`

Note: The additional extension `.bak` is sometimes used for backups.

Globbing

The globbing asterisk `*` is used for expanding to every possible path in a directory.

It is best explained using examples:

- `cat *.sh` prints the content of all files ending with `.sh` in the current directory.
- `mv dir1/* dir2` moves all **visible** files and directories from `dir1` to `dir2`.
- `mv dir1/.* dir2` moves all **hidden** files and directories from `dir1` to `dir2`.
- `mv dir1/{,.}* dir2` expands to `mv dir1/* dir1/.* dir2` and therefore moves all *visible and hidden* files and directories from `dir1` to `dir2`.

Note: Fish can expand a globbing when pressing `Tab` after an asterisk `*` (but it is not always helpful).

Regex

Regular **ex**pressions specify a match patter in a text.

They can be used for example with `grep`, `rg`, `find`, `fd`, `vim`, etc.

Similar expressions are also used for Git (`.gitignore`) and containerization (`.containerignore`) which we will learn about later.

Here are some of the most important building blocks:

- `^` : Start of line
- `$` : End of line
- `()` : Group
- `|` : Alternation
- `[abcd]` : Character set, here `a` until `d`
- `[a-z]` : Character range, here `a` until `z`
- `[^b-h]` : Negated character range, here `b` to `h`
- `.` : Any character
- `.*` : 0 or more characters
- `.+` : 1 or more characters
- `\w` : Letter or number
- `\W` : Neither letter nor number
- `\d` : Digit
- `\D` : Not digit
- `\s` : Whitespace
- `\S` : Not whitespace

Writing regular expressions is not easy, but there are many (online) tools that you can use to test and debug your regex.

CLIs of the day

cargo

Cargo is the package manager of the programming language Rust.

You don't have program in Rust to use it. It is listed here because you can use it to install many of the fancy command line programs written in Rust (like `zellij`, `ripgrep`, `fd-find`, `bat`, `tealdeer`, etc.).

You can install a program with cargo using the command:

```
cargo install PROGRAMNAME
```

You don't need to run it with `sudo` since it installs programs for the current user only. It doesn't modify files not belonging to the user.

To update programs installed using `cargo`, you need to have `cargo-update` installed:

```
# The package `openssl-devel` is needed to compile `cargo-update`
sudo dnf install openssl-devel

# To be able to run cargo install-update -a
cargo install cargo-update

# Update installed crates
cargo install-update -a
```

curl

We did already use `curl`, but not yet for downloading.

```
# Download a file into the current directory while keeping the default name of the
file.
curl -L LINK_TO_FILE -O

# Download a file while giving the path to save the file into
# (notice that we are using small o now, not O)
curl -L LINK_TO_FILE -o PATH
```

`-L` tells `curl` to follow redirections (for example from `http` to `https`).

xargs

`xargs` uses each line from the standard input (stdin) as an argument to the command specified after it.

Here is an example that shows the content of all files with the extension `.txt` in the current directory.

```
ls *.txt | xargs cat
```

If you have the files `file1.txt` and `file2.txt` in the current directory, then the command above is equivalent to just running `cat file1.txt file2.txt`.

ripgrep

ripgrep is like `grep`, but it offers many additional features and has much better performance (+ it is written in Rust 🦀).

Here is an example of how you can use it with regex to catch a group:

```
$ cat demo.csv
a,b,c
x,y,z
1,2,3

$ rg '.*,(.*),.*' -r '$1' demo.csv
b
y
2
```

Tasks

Organize the files and directories of your tasks in separate directories!

Task: Cargo 📦

Use `cargo` to install the following crates:

- `cargo-update`
- `tealdeer`

It might take a long time to compile everything.

`cargo-update` should be installed to be able to run `cargo install-update -a` to update all installed crates. Try running the command. But you should not find any updates since you did just install the crates.

The crate `tealdeer` provides you with the program `tldr`.

Run `tldr --update`. Then run the following two commands:

```
tldr dnf
tldr apt
```

It should be obvious to you what `tldr` does after you run the commands above and read their output. Try it with other programs than `dnf` and `apt`!

🟢 Run `cs task cargo`.

Task: Fish configuration

Disable the default greeting by Fish everytime you start it.

The configuration file should be `~/.config/fish/config.fish`.

🟢 Run `cs task fish-config`.

Task: Fish in Zellij

Configure Fish as the default Zellij shell.

- Run `cs task fish-in-zellij`.

Task: Curly line count.

Use `curl` to fetch this file: https://codeberg.org/mo8it/collective-score/raw/commit/4ff0cd6f871e4a17a7ecd36d7d01ca7713c11ca1/Cargo.toml

You don't have to save it to disk!

Now, pipe the output of `curl` into `wc` (with some option) to count the number of characters.

- Pipe the number of characters into `cs task curly-wc`.

Hints

- `curl … | … | cs task curly-wc`

Task: IO redirections

I placed a program called `mixed-output-generator` on your system.

If you run it, it will create the file `dont_overwrite.txt` in your current directory and output random text to both stdout and stderr.

First, run it while writing its standard output to the file `normal_output.log` and its standard error to the file `errors.log`.

After that, run it again while writing both standard output and standard error to the same file `verbose.log`.

Now, run it for the last time while *appending* the standard output to the file `dont_overwrite.txt` that it creates.

- Run `cs task redirections`.

Task: Gimme that PDF 😈

The following website uses a PDF file but it doesn't let you download it: https://knowunity.de/knows/biologie-neurobiologie-1c6a4647-4707-4d1b-8ffb-e7a750582921

Now that you are kind of a "hacker", you want to use a workaround 😈

Use `curl`, pipes `|`, `rg` (ripgrep) and `xargs` to parse the HTML of the web page, extract the link to the PDF file and download it. Save the PDF file using the name `knowunity.pdf`.

The link to the PDF file starts with `https://` and ends with `.pdf`.

🟢 Run `cs task pdf` in the same directory where the downloaded PDF file is.

Hints

- To capture a regex group using ripgrep, you should use `rg '.*(GROUP_PATTERN).*' -r '$1'` after replacing `GROUP_PATTERN` with the pattern that you are looking for.
- If you find the link of the file using `rg`, copy it into a browser to make sure that you got a correct link to a PDF file.

Task: Vim game

In this task, we are going to play a game! 🎮

The game is an educational game that lets you learn the very basics of navigation in Vim which we learn about later. But for now, I can get familiar with the basics during the game.

Play the game on this website: https://vim-adventures.com

Day 3

This day is all about Git and it will be very interactive.

I will present the content live while drawing on the board, running commands, using graphical user interfaces and git platforms.

If you are reading this from home, then I am sorry for not having explanations here. I think that Git can be explained better live. But don’t worry, if you do the tasks then you will have a solid knowledge about Git. Especially if you finish big parts of the game Oh My Git!

Notes

The participants of the course will add a description to each of these Git subcommands.

git config

Configure user-specific information like name, email etc. An example for setting the email is

```
git config --global user.email you@gotthis.com
```

An example for setting the username would be

```
git config --global user.name "Wonderful-Person"
```

git init

git clone

git status

git add

This command stages one or more files for the next commit, adding them to everything that is committed.

Examples are:

```
git add "example.txt"
```

or

```
git add --all
```

if you want to stage all changes.

git commit

This command is used to record the changes you've made to the repository as a new commit. A commit is a snapshot of the changes you've staged (using `git add`) and is accompanied by a commit message that describes the purpose of the changes you are committing.

Usage of `git commit`:

```
git commit -m "Your commit message here"
```

git log

git diff

git branch

git checkout

This command enables you to switch to another branch or to another commit.

Examples are:

```
git checkout "branchname"
```

or

```
git checkout -b "branchname"
```

If you want to create a new branch and switch to it.

git push

git pull

git rebase

git merge

This command is used to combine two branches together. Use git checkout to visit the branch you want to merge to.

```
$ git checkout main
$ git merge branch2
```

This will merge the branch “branch2” into the branch, where this command was executed in. (Here: “main”)

git reset

git revert

git cherry-pick

Description

The `git cherry-pick` command applies a specific commit from one branch to another. This can be useful for undoing changes, applying bug fixes to a different branch, or picking out specific changes from a merge commit.

Usage

The basic syntax for the `git cherry-pick` command is:

```
git cherry-pick <commit-hash>
```

where `<commit-hash>` is the SHA-1 hash of the commit you want to cherry-pick. You can also use the `-n` option to preview the changes without actually applying them.

For example, to cherry-pick the commit with the SHA-1 hash `1234567890abcdef`, you would run the following command:

```
git cherry-pick 1234567890abcdef
```

Example with visualization of the git log graph:

- Let's assume the current branch state:

```
a - b - c - d   Main
     \
       e - f - g Feature
```

- Ensure that we'are working in the `main` branch:

```
git checkout main
```

- Then let's execute the following `cherry-pick` command:

```
git cherry-pick f
```

- Once executed, branch state will change to:

```
a - b - c - d - f   Main
     \
       e - f - g Feature
```

Options

The `git cherry-pick` command has a few options that you can use to customize its behavior:

- `-n` : Preview the changes without actually applying them.
- `-e` : Edit the commit message before applying it.
- `-s` : Sign the commit with your GPG key.
- `-S[<keyid>]` : Sign the commit with the specified GPG key.

git remote

git blame

git stash

git tag

git lfs

CLIs of the day

typos

A command line tool that checks your (code) files for typos.

You can just run it in the directory that you want to check and it will show you all the typos it finds.

You can then run `typos -w` to let it fix typos automatically. But you have to fix some typos yourself if `typos` has more than one possible fix.

gitui

A terminal user interface (TUI) for Git written in Rust 🦀 It is very user friendly and self explanatory like Zellij.

lazygit

Another TUI for Git (but written in Go 👎). Try both to find out which one you like more.

git-cliff

Automatically generates a `CHANGELOG.md` file based on conventional commits.

Conventional commits have their pros and cons. Try them out to see if you like their systematic approach.

Tasks

Task: Installation

Install Git and VS-Code on your own machine (not in the provided browser terminal).

Open a Git terminal and run the following commands to tell Git about your name (`USERNAME`) and email (`EMAIL`)

```
$ git config --global user.name "USERNAME"
```

```
$ git config --global user.email "EMAIL"
```

You don't have to use VS-Code for the tasks, but you should at least try it out with its Git integration.

Task: Register on Codeberg 🏔️

Register an account on Codeberg. It is a Git platform similar to Github and Gitlab, but it is fully free and open source and based in Europe.

Task: Stars ⭐

Just to learn how to star repositories, you could add some stars to my personal ones 😜

We do everything here for educational purposes 😇

Task: Issues

Create an issue on the repository of the course that gives me some quick feedback about the course so far.

Read some of the issues created by others and add some emoji reactions and maybe a reply to them.

Task: SSH key pair 🔑

Create an SSH key pair *without a passphrase* and add its public part to your Codeberg account following the beginning of this guide.

I am recommending to not use a passphrase for now to be able to focus on Git itself. But for a better security in general, you should use a passphrase for your SSH keys outside this course.

Task: Markdown

You can take a look at this guide for this task.

Create a new repository called `dev-tools-demo` and clone it to your machine with `git clone SSH_GIT_LINK`.

After cloning, a new directory should be created with the name of the repository. Move into that directory and create the markdown file `README.md`.

Follow this guide about Markdown to write some words about the course so far in the README file. Use all of the Markdown following building blocks:

- Headings with different levels
- Paragraphs with some **bold** and *italic* text
- A list
- A table
- A blockquote
- Some `bash` code examples inside of code blocks
- A link to some website
- An image

Create a commit after each building block:

```
$ git add README.md
```

```
$ git commit -m "MEANINGFUL MESSAGE"
```

`push` your changes after being done with all building blocks.

Verify that the README file was rendered as expected. If not, fix the Markdown issues, commit and push the changes again.

Task: License 📄

Add the AGPLv3 license as a file `LICENSE.txt` to your demo repository. Commit and push the file!

You can read more about licenses later. Discussing different ones is out of the scope of this course.

But it is important to add a license to your projects! AGPLv3 is the one that I normally use and recommend for free open source software. You can read about it and compare it to others using choosealicense.com for example.

Task: Write the script for me 😜

Fork the repository of the course and clone your fork into your machine.

You will find the most important Git subcommands in the file `src/day_3/notes.md`, but these are still missing a description.

Pick one of the subcommand that don't have a description yet. Create a branch that has the same name as the chosen Git subcommand.

Add a description to the chosen subcommand in your own words. Add some examples for how to use that subcommand.

Commit your changes and push the branch to your fork. Create a pull request using that branch.

Task: Collaboration

> Note: You have to skip this task if you are working from home.

Team up with another participant that is also done with his demo repository and do the following together:

- Give the other participant a star ⭐
- On the repository of the other participant, create an issue giving some **nice** feedback about the content of the README file.

- Reply to the issue created by the other participant on your own repository and close it if no actions are required.
- Fork the other repository, clone the fork, create a new branch and add a few lines to the README file that fit the original content.
- Push that branch to your fork.
- Create a pull request on the original repository using the pushed branch.
- Merge the pull request that you receive on your own repository.

Task: Oh My Git!

Download and play the awesome educational game Oh My Git! 🎮

Day 4

Today, you will learn to write scripts for task automation.

We will focus on shell scripting in Bash. But at the end, scripting in Python is presented briefly as a more flexible scripting option in case your scripts get complicated.

But before we start writing our first scripts, we will learn two command line editors that offer many more features than `nano` and can make editing files much more efficient.

Advanced terminal editors

`nano` is fine if you just want to occasionally edit some files in the terminal.

But if you want to be able to edit files more efficiently, then you should consider Vim and Helix.

Vim

Vim is a very popular *modal* terminal editor.

Modal means that it has multiple editing modes allowing you to use your keyboard not only to type text but also to navigate and manipulate it.

Modal editing allows you to edit files without needing to touch the mouse. Being able to work without the mouse is not only good for the terminal, but it also allows you to edit code faster because you keep your hands on the keyboard and have a keybinding for everything.

Each mode has its keybindings. When you just enter Vim, then you are in the **normal mode** which has the following most important keybindings:

- `:q` : Quit (**very important!**)
- `:wq` : Write buffer (save) and exit (**very important!**)
- `:q!` : Quit without saving (**very important!**)
- `j` : Down
- `k` : Up
- `h` : Left
- `l` : Right
- `i` : Insert at left of cursor
- `a` : Insert at right of cursor (append)
- `I` : Insert at beginning of line
- `A` : Append to end of line
- `Esc` : Normal mode
- `w` : Go to beginning of next word
- `b` : Go to beginning of last word
- `e` : Go to end of word
- `gg` : Go to beginning of file
- `G` : Go to end of file
- `0` : Go to beginning of line
- `$` : Go to end of line

- `%` : Go to the other bracket
- `u` : Undo
- `Ctrl+r` : Redo
- `:h` : Help
- `:w` : Write buffer
- `/PATTERN` : Search
- `n` : Next match
- `N` : Previous match
- `*` : Next match of the word under cursor
- `o` : Add line below and enter insert mode
- `O` : Add line above and enter insert mode
- `v` : Start selection
- `V` : Block selection
- `y` : Yank (copy)
- `p` : Paste
- `x` : Delete one character
- `dw` : Delete word
- `dd` : Delete line
- `D` : Delete util end of line
- `cw` : Change word
- `cc` : Change line
- `C` : Change until end of line
- `di(` : Delete inside bracket `(` . Can be used with other brackets and quotation marks.
- `da(` : Same as above but delete around, not inside.
- `ci(` : Change inside bracket `(` . Can be used with other brackets and quotation marks.
- `ca(` : Same as above but delete around, not inside.
- `:%s/OLD/NEW/g` : Substitute `OLD` with `NEW` in the whole file (with regex)
- `:%s/OLD/NEW/gc` : Same as above but ask for confirmation for every substitution
- `:N` : Go line number `N`
- `.` : Repeat last action
- `<` and `>` : Indentation
- `q` : Start recording a macro (followed by macro character)

When you press `i` for example, then you enter the **insert mode** where you can type text.

There is also the **visual mode** which you can enter by pressing `v` in the normal mode. It allows you to select text first to then act on it.

You can exit the insert or visual mode by pressing the escape key `Esc` .

You might think that it is complicated. You are right, it is complicated when you start using it. You need to practice it for some time, but after that, the keybindings become a second nature

of editing for you. You don't think about which keybindings to press anymore. You just get the muscle memory and can focus on your editing flow.

Should you use Vim? I would only recommend using it if you are editing some configuration files or small scripts in the terminal (on a server for example). But for programming, you should use an editor that offers more features that support you while programming. It should at least have LSP (language server protocol) support.

The easiest option to get started with for programming on your own machine is VS-Code. For a more advanced option, the next section will present another modal terminal editor that has even more features out of the box compared with Vim.

So why do we bother learning Vim then? Although you don't have to use Vim, you should at least learn it and be familiar with its basic keybindings. Not only to be able to use it when you have to, but also because Vim keybindings are widely used by other programs, especially terminal user interfaces.

Do you remember how we learned to navigate the manual when using `man`? Well, many of the keybindings that it supports are inspired by Vim! Examples are exiting with `q` or `:q` and searching with `/`, `n` and `N`. But you can also navigate the manual using `j`, `k`, `l`, `h` instead of the arrow keys! If you press `h` in some manual, then you see all keybindings that you can use and you will be surprised by how many are similar to Vim's.

> **Note**: The keybindings presented when pressing `h` are those of `less` which is a so called terminal pager
>
> It is used by other programs too, not only `man`. You can also use it yourself by piping a command that produces some long output into it.
>
> Try running `curl -s https://dev-tools.mo8it.com` and then `curl -s https://dev-tools.mo8it.com | less` to see the difference.

You can even use Vim keybindings in other editors to boost your editing efficiency! 🚀 In VS-Code for example, you get Vim keybindings by installing a plugin like this.

> **Note**: Vim is the successor of `vi`. If you are using a system that doesn't have `vim` preinstalled and you don't have superuser privileges to install it, then you will most probably at least find `vi` installed.

Helix

Vim is the most popular modal text editor, but it is old. If you want to have a modal terminal text editor that can support you when coding, then try out Helix!

It is a modern editor written in Rust with awesome features that you can read about on its website. The most important one is the language server support which gives you completions, code actions, function signatures, language aware code navigation etc. You can watch this video for an introduction to it.

Most of its keybindings are similar to these of Vim. But it works by a selection followed by a verb like `wd` (word delete) instead of a verb followed by selection like `dw` (delete word) in Vim. This makes editing more intuitive since you can see what will be changed before the change happens.

Personally, I use Helix for all my text editing including programming and working on servers. Helix combined with Zellij is a powerful workspace where you can do everything efficiently in the terminal. It has a steep learning curve though. VS-Code (maybe with Vim keybindings) can often be enough 😉

Shell scripting

Task automation requires multiple instructions that have to run on demand. To combine multiple commands together for automation, we need to write a shell script.

Since Bash is the default shell on most Linux distributions, we will learn Bash scripting.

Although I recommend using the Fish shell when using the terminal and although Fish supports scripting too, I would not recommend using it for scripting since you would not be able to share and run these scripts anywhere. You need to have Fish installed which is not always the case.

I use Fish for my personal scripts. But if I write a script that will be shared with others, then I write it in Bash. Learn to write Bash scripts first before considering using Fish scripts, even for your personal scripts.

First Bash script

Let's write our first Bash script:

```
#!/usr/bin/bash

echo "What is your favorite operating system after reading this book?"
echo "1. Linux"
echo "2. Windows"
echo "3. Mac"

RIGHT_ANSWER=1

# `-p` Sets the prompt message
read -p "Enter a number: " ANSWER

if [ $ANSWER == $RIGHT_ANSWER ]
then
    echo "Good choice!"
else
    # Any answer other than 1
    echo "Nah, that can't be right! It must be an error!" 1>&2
fi
```

Copy this code into a file called `which-os.sh`.

Now, run `chmod u+x which-os.sh`. Then run `./which-os.sh`. Don't worry, everything will be explained afterwards.

After running the script, you will see a prompt asking you to enter a number corresponding to an operating system. If you choose Linux, you get the output “Good choice”. This output is in the standard output.

If you don’t choose Linux, you get the output “Nah, that can’t be right! (...)”. This output is redirected to the standard error.

The building blocks of the script above will be explained in the next sections.

Shebang

The first line of our first script starts with `#!` which is called the *shebang*. The shebang is followed by the program that runs the script. Since the script is a Bash script, we use the program `bash` to run it.

But writing `bash` after the shebang is not enough. We have to specify the full path to the program. We can find out the path of a program by using the command `which`:

```
$ which bash
/usr/bin/bash
```

You can also write a Python script and add a shebang at its beginning. We can find out the path to the Python interpreter by running the following:

```
$ which python3
/usr/bin/python3
```

This means that we can now write this script:

```
#!/usr/bin/python3

print("Hello world!")
```

Let’s save this tiny Python script as `hello_world.py`, make it executable with `chmod` (will be explained later) and then run it:

```
$ chmod u+x hello_world.py

$ ./hello_world.py
Hello world!
```

We could have written the Python script without the shebang, but then we would have to run with `python3 hello_world.py`. Adding the shebang lets you see a script as a program and ignore what language it is written in when running it.

You can use the shebang with any program that can run a script.

Variables

In our first Bash script, we have the line `RIGHT_ANSWER=1`. This line defines a variable with the name `RIGHT_ANSWER` and the value `1`.

To define a variable in Bash, you have to write the name of the variable **directly** followed by an equal sign `=`. The equal sign has to be **directly** followed by the value of the variable.

directly followed by means that spaces between the variable name, the equal sign `=` and the value are not allowed!

But what if you want to set a variable equal to a string that contains spaces?

In that case, you have to use quotation marks `"`. For example:

```
HELLO="Hello world!"
```

To read the value of a defined variable, we use a dollar sign `$` before the variable name. For example:

```
echo $HELLO
```

The line above would output `Hello world!`.

You can use defined variable inside a variable definition:

```
MESSAGE="Tux says: $HELLO"
echo $MESSAGE
```

The two lines above would lead to the output `Tux says: Hello world!`.

Capturing command output

You can capture the standard output of a command using the the syntax `$(COMMAND)`. Example:

```
BASH_VERSION=$(bash --version)
```

The line above saves the output of the command `bash --version` in the variable `BASH_VERSION`.

Let's run the command in the terminal first to see its output:

```
$ bash --version
GNU bash, version 5.2.15(1)-release (x86_64-redhat-linux-gnu)
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>

This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Now, let's output the variable that we did define above:

```
$ echo $BASH_VERSION
GNU bash, version 5.2.15(1)-release (x86_64-redhat-linux-gnu) Copyright (C) 2022
Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later
<http://gnu.org/licenses/gpl.html> This is free software; you are free to change
and redistribute it. There is NO WARRANTY, to the extent permitted by law.
```

You can see that the lines are squashed into one line! If you want to output the lines without them being squashed, you have to use quotation marks `"`:

```
$ echo "$BASH_VERSION"
GNU bash, version 5.1.16(1)-release (x86_64-redhat-linux-gnu)
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>

This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

This is the output that we expect 😀

Environment variables

Let's write the following tiny script `hello.sh`:

```
#!/usr/bin/bash

echo "Hello $USER!"
```

Now, we run the script. The output on my machine is:

```
$ chmod u+x hello.sh

$ ./hello.sh
Hello mo8it!
```

We see `Hello mo8it!` as output. This is because my user name on my machine is `mo8it`. `USER` is a so called **environment variable** that is defined for all programs. If you run the script on your machine, you will get your username instead of `mo8it`.

Now, let's run the following:

```
$ USER=Tux ./hello.sh
Hello Tux!
```

We defined an environment variable `USER` with the value `Tux` just before running the script. This variable overwrites the global value of the variable `USER` in our script. Therefore we get the output `Hello Tux!` and not our user name.

You can use environment variables not only to overwrite existing ones variables. These are basically variables that can be used by the program that you specify after their definition.

Comments

In our first Bash script, you can find three lines starting with a hashtag `#`. Two lines are comments, the shebang `#!` is an exception as a special comment at the beginning of a file that is not ignored while running the script. All other lines that start with `#` are comments that are ignored by the computer. Comments are only for humans to explain things.

Especially in long scripts, you should write comments to explain what the script itself and its "non trivial sections" do.

User input

In a script, you can ask for user input. To do so, you can use the command `read`.

In our first Bash script, we use `read` to ask the user for his answer. The input is then saved in the variable `ANSWER` (you can also choose a different name for this variable). After the line with `read`, you can use the variable storing the input just like any other variable.

Arguments

To read the `n`-th argument that is provided to a script, we can use `$n`.

Take a look at the following example script called `arg.sh`:

```
#!/usr/bin/bash

echo "The first argument is: $1"
```

When you run this script with an argument, you get the following output:

```
$ ./arg.sh "Hello"
The first argument is: Hello
```

Conditions

`if` block

Our first Bash script checks if the user input which is stored in the variable `ANSWER` equals the variable `RIGHT_ANSWER` which stores the value `1`.

To check for a condition in Bash, we use the following syntax:

```
if [ CONDITION ]
then
    (...)
fi
```

Here, `(...)` stands for the commands that we want to run if the condition is true.

In our first Bash script, we check for equality of two variables with a double equal sign `==`.

`fi` is not a typo! It is just `if` reversed to indicate the end of the `if` block. Although the syntax is not the best, you have to sadly accept it. Bash does not have the best syntax...

Speaking about syntax: You have to take spaces seriously with conditions.

For example, if we define the variable `VAR=1`, the following snippets **do not work** (or have an unexpected behavior):

1. No space after `[`

```
if [$VAR == 1 ]
then
    echo "VAR has the value 1"
fi
```

2. No space before `]`

```
if [ $VAR == 1]
then
    echo "VAR has the value 1"
fi
```

3. No space before `==` but a space after `==`

```
if [ $VAR== 1 ]
then
    echo "VAR has the value 1"
fi
```

4. No space after `==` but a space before `==`

```
if [ $VAR ==1 ]
then
    echo "VAR has the value 1"
fi
```

5. No space before `==` and after `==`

```
if [ $VAR==1 ]
then
    echo "VAR has the value 1"
fi
```

But the following snippet **works**:

- Space after `[` , before `]` , before `==` and after `==`

```
if [ $VAR == 1 ]
then
     echo "VAR has the value 1"
fi
```

`else` block

The `else` block runs commands inside it only if the `if` condition is false. The syntax is:

```
if [ CONDITION ]
then
    # Runs only if CONDITION is true
    (...)
else
    # Runs only if CONDITION is false
    (...)
fi
```

Example:

```
if [ $VAR == 1 ]
then
    echo "VAR has the value 1"
else
    echo "VAR does not have the value 1"
fi
```

Python scripting

Bash scripts are fine if your script is not complicated. But Bash is ugly and if you try for example to use arrays/lists in Bash, you probably should just use Python for what are trying to accomplish with your Bash script.

You can use what you have learned about running programs and feed Python with commands that it can run:

```
>>> import subprocess

>>> proc = subprocess.run(["ls", "student"])
(…)
CompletedProcess(args=['ls', '/home/student'], returncode=0)

>>> proc.returncode
0

>>> proc = subprocess.run("ls /home/student", shell=True)
(…)
CompletedProcess(args='ls /home/student', returncode=0)

>>> proc = subprocess.run("ls /home/student", shell=True, capture_output=True,
text=True)
CompletedProcess(args='ls /home/student', returncode=0, stdout='(…)', stderr='')

>>> proc.stdout
(…)

>>> proc.stderr

>>> proc = subprocess.run("ls /home/nonexistent", shell=True, capture_output=True,
text=True)
CompletedProcess(args='ls /home/nonexistent', returncode=2, stdout='', stderr="ls:
cannot access '/home/nonexistent': No such file or directory\n")

>>> proc = subprocess.run("ls /home/nonexistent", shell=True, capture_output=True,
text=True, check=True)
---------------------------------------------------------------------------
CalledProcessError                        Traceback (most recent call last)
(…)
```

The most used modules when writing system scripts with Python are `pathlib`, `os`, `shutil`.

```
>>> from pathlib import Path
>>> p = Path.home() / "day_5"
PosixPath('/home/student/day_5')

>>> p.is_dir()
(…)

>>> p.is_file()
(…)

>>> p.exists()
(…)

>>> p.chmod(0o700)
(…)

>>> rel = Path(".")
PosixPath('.')

>>> rel.resolve()
PosixPath('/home/student/(…)')
```

CLIs of the day

cut

Demo file `demo.txt`:

```
here,are,some
comma,separated,values
de mo,file,t x t
```

```
# Get the N-th column by using SEP as separator
cut -d SEP -f N FILE
```

Example:

```
$ cut -d "," -f 1 demo.txt
here
comma
de mo
```

You can also pipe into `cut` instead of specifying `FILE`.

sed

```
# Substitute
sed 's/OLD/NEW/g' FILE

# Delete line that contains PATTERN
sed '/PATTERN/d' FILE
```

Example:

```
$ sed 's/values/strings/g' demo.txt
here,are,some
comma,separated,strings
de mo,file,t x t

$ sed '/separated/d' demo.txt
here,are,some
de mo,file,t x t
```

You can also pipe into `sed` instead of specifying `FILE`.

When you specify `FILE`, you can use the option `-i` to operate *inplace*. This means that the file is modified directly.

find

Find everything ending with `.sh` of type file (`f`) using globbing:

```
find . -type f -name '*.sh'
```

Using regex:

```
find . -type f -regex '.+\.sh'
```

Tasks

Don’t forget to add a shebang to your scripts and make them executable!

Task: Essential tools

Write a Bash script that installs the following *essential* Linux packages for you:

- `cmatrix`
- `cowsay`
- `lolcat`
- `cbonsai`

After the installation is done, the script should let `cowsay` inform the user that the installation was successful.

Test your script!

Fun: Run the command `cbonsai -i -l -t 0.01 -w 1` 🌳

Task: Current directory

Write a Bash script that calculates the total number of files and directories (together) in the current directory. For example, if the current directory has two directories and one file, then the number would be `3`. Store the result into a variable.

The output should be the full path to the current working directory with a message telling about the calculated number. Example:

```
$ whereami.sh
You are in the directory /home/student/scripts
There are 3 files and directories in this directory.
```

Hints

- `ls -1` shows every file or directory in the current directory on a separate line.

Task: Backup then update

Write a Bash script that does the following:

1. Create the directory `~/installed_packages` if it does not exist. If it exists, it should not through any errors.
2. Stores the output of the command `dnf list --installed` into the file `~/installed_packages/dnf.txt`.
3. Stores the output of the command `cargo install --list` into the file `~/installed_packages/cargo.txt`.
4. Installs system upgrades using `dnf`.
5. Installs Cargo updates.
6. Check if `cowsay` is installed. Only if not, install it first. After that, `cowsay` should inform you that installed packages where logged into the directory `~/installed_packages` and updates were successfully installed.

If any of the steps above fails, the script should stop and not continue. This can be achieved by adding the following line to the beginning of the script (after the shebang): `set -e`.

Task: Scripted “Gimme that PDF” 😈

On day 2, you downloaded a PDF given some web page link. We want to write a script now that is generic over the web page link.

It should ask the user for a web page link and then download the PDF if a PDF link was found on that web page.

The script should exit with the status code 1 while printing an error message to stderr in case no PDF link was found on the web page ❌

Otherwise, it should print a message to stdout that the download was successful ✔️

Task: Job scheduler

> **Warning** ⚠️ : This task is not an easy one. Don't give up quickly and ask for help if you don't get further!

In this task, we want to write our own job scheduler.

Understanding how job schedulers work is important when you are working on a computer cluster.

Computer clusters are shared by many users. Therefore, running jobs on a cluster has to be scheduled to make sure that the resources are shared probably.

In this task, we will keep it simple. No aspects of multiple users or any optimizations.

We want to be able to submit a job as a single script (without any dependencies). The submitted scripts should run one after the another.

We will use the program `inotifywait`. This program can monitor a directory and notify on changes within this directory.

1. Start Zellij if you are not already using it.
2. Find out which package provides the program `inotifywait` and install it.
3. Read the manual of `inotifywait` for a better understanding of what it does.
4. Find out what events mean in the context of `inotifywait`.
5. Find out how to tell `inotifywait` to keep monitoring a directory and not exit after the first event.
6. Create a new directory called `jobs` to be monitored.
7. Create a new directory called `logs` that will be used later.
8. Run `inotifywait` while telling it to monitor the directory `jobs`. Leave the command running in one Zellij pane and open a second pane to continue the work in.
9. Create a file **outside** of the directory `jobs` and then copy it to the directory `jobs`.
10. Look at the output of `inotifywait` in the first pane after copying the file in to the `jobs` directory.
11. Based on that output, choose an event that you want to listen to with `inotifywait` that tells you when a file is **completely written** to the directory `jobs`. Use the manual to read more about specific events.
12. Find an option that lets you tell `inotifywait` to only notify when the chosen event occurs.
13. Find an option that lets you format the output of the notification of `inotifywait`. Since we only listen on one event and monitor only one directory, an output that shows **only**

the name of the new file should be enough.

14. Enter the `inotifywait` command that you have until now in a script. Now, extend it by using a `while` loop that continuously listens on the notifications of `inotifywait`. Use the following snippet after replacing `…`:

```
inotifywait … | while read NEW_SCRIPT_NAME
do
    …
done
```

15. After a notification, the body of the `while` loop should first print the name of the script that was added (`NEW_SCRIPT_NAME`). Then, the new script should be run.
16. Save the standard output and standard error of the script into two separate files in the `logs` directory. If the name of the script is `job.sh` for example, then the output should be in the files `logs/job.sh.out` and `logs/job.sh.err`.

Hints

- Take care of file permissions.

Task: Job submitter

In this task, we will write a small script called submitter that lets us submit a job script to the scheduler from the last task.

The submitter should take the path to the job script as a single required argument.

The submitter should then copy the job script to the directory `jobs` while adding the current time and date as a prefix to the new job script name.

Read the manual of the command `date` to know how to get the current time and date in the following format: `2022-08-22T20:00:00+00:00`.

If the name of the job script is `job.sh` for example, the job script should be named `2022-08-22T20:00:00+00:00_job.sh` in the `jobs` directory.

Use variables while writing the script to make it more understandable.

Hints

- To save the output of a command into a variable, you have to use the following syntax after replacing `…` :

```
DATETIME=$(date …)
```

Task: Submit a job

Write a small job script that requires at least 10 seconds to run (can be simulated with the command `sleep 10`). Submit that job script by using the submitter from the last task.

Make sure that the scheduler is running in one Zellij pane before submitting.

Submit your job script multiple times and take a look at the pane where scheduler is running to make sure that the job scripts are running one after the other.

Verify the redirection of the standard output and standard error in the directory `logs` .

Day 5

In this day, we will learn how to connect to servers using SSH and work on them.

We will also learn how to use containers in Linux so that will be able to host your own website at the end!

Notes

SSH

Setup host

In `~/.ssh/config`

```
Host HOST
    HostName SERVERIP
    User SERVERUSER
```

Generate key pair

```
ssh-keygen -t ed25519 -C "COMMENT"
```

Leave blank to take default for the prompt `Enter file in which to save the key (/home/USERNAME/.ssh/id_ed25519)`.

Then, you can enter a passphrase for your key. To keep it simple while learning, we will not add one now. Press enter twice to not use a passphrase. **But you should use a passphrase when you work with real servers!**

Add the public key to the server

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub HOST
```

Connect

```
ssh HOST
```

Config on server

Very important for security! Only after adding the public key to the server!

> WARNING ⚠️ :
>
> Verify that you are only asked for the passphrase of the SSH key before continuing in this section!
>
> If you are asked for the password of the user on the server when connecting, then the authentication with a key did not work. Therefore, don't set `PasswordAuthentication no`! Fix the issue with the key authentication first. **Otherwise, you will be locked out of the server!** ⚠️

In `/etc/ssh/sshd_config` on the server:

Uncomment line with `PasswordAuthentication` and set it to `PasswordAuthentication no`

Save and exit, then run:

```
sudo systemctl restart sshd
```

If you are locked out after running this command, then you did not take the warning above seriously!

Rsync

From server:

```
rsync -Lahz HOST:SRC_PATH DEST_PATH
```

To server:

```
rsync -Lahz SRC_PATH HOST:DEST_PATH
```

Options:

- `-a`, `--archive`: Set of useful options to preserve permissions, use recursive mode, etc.
- `-h`, `--human-readable`: Output number in a human-readable format.
- `-z`, `--compress`: Use compression.

- `--partial` : Continue after interruption.
- `-L , --copy-links` : Copy links.
- `-v , --verbose` : Show more infos.
- `--delete` : Delete files from `DEST_PATH` if they don't exist on `SRC_PATH` anymore. **Use with caution!!!**

Systemd

Check status of a service:

```
sudo systemctl status SERVICENAME
```

Enable service:

```
sudo systemctl enable SERVICENAME
```

Start service:

```
sudo systemctl start SERVICENAME
```

Enable and start service at the same time:

```
sudo systemctl enable --now SERVICENAME
```

Disable service:

```
sudo systemctl disable SERVICENAME
```

Stop service:

```
sudo systemctl stop SERVICENAME
```

Disable and stop service at the same time:

```
sudo systemctl disable --now SERVICENAME
```

Firewalld

Install and enable firewalld:

```
sudo dnf install firewalld
sudo systemctl enable --now firewalld
```

View open ports and services:

```
sudo firewall-cmd --list-all
```

Open ports 80 (http) and 443 (https):

```
sudo firewall-cmd --add-service http
sudo firewall-cmd --add-service https
sudo firewall-cmd --runtime-to-permanent
```

or:

```
sudo firewall-cmd --add-port 80/tcp
sudo firewall-cmd --add-port 443/tcp
sudo firewall-cmd --runtime-to-permanent
```

Podman

```
# Search for image
podman search python

# Pull image
podman pull docker.io/library/python:latest

# See pulled images
podman images

# Run container and remove it afterwards
podman run -it --rm docker.io/library/python:latest bash

# Create network
podman network create NETWORKNAME

# Create container
podman create \
    --name CONTAINERNAME \
    --network NETWORKNAME \
    -e ENVVAR="Some value for the demo environment variable" \
    --tz local \
    docker.io/library/python:latest

# Start container
podman start CONTAINERNAME

# Enter a running container
podman exec -it CONTAINERNAME bash

# Stop container
podman stop CONTAINERNAME

# Generate systemd files
podman generate systemd --new --files --name CONTAINERNAME

# Create directory for user's systemd services
mkdir -p ~/.config/systemd/user

# Place service file
mv container-CONTAINERNAME.service ~/.config/systemd/user

# Activate user's service (container)
systemctl --user enable --now container-CONTAINERNAME
```

Keep user's systemd services live after logging out:

```
sudo loginctl enable-linger USERNAME
```

Options:

- `-v, --volume: SRC_PATH:DEST_PATH:L`. Label should be one of `z`, `z,ro`, `Z` or `Z,ro`.
- `-p, --publish: SERVER_PORT:CONTAINER_PORT`

Tasks

Task: SSH 🔑

Generate an SSH key pair in the browser terminal if you did not do so on day 3 yet. Send me the public key per email: mo8it@proton.me

The public key has the extension `.pub`. Don't send me the private key!!! **You should never send your private SSH keys to anyone!**

I will then append your public key to `~/.ssh/authorized_keys` on the server that we will use in the next tasks. After I add your public key, you will be able to login to the server and do the next tasks.

Create the file `~/.ssh/config` if it does not exist and add the server as a host with the name `linux-lab`:

```
Host linux-lab
  HostName 38.242.215.155
  User admin
```

After that I add your public key, connect to the server using the host name that you entered in `~/.ssh/config` which is `linux-lab`:

```
ssh linux-lab
```

Task: User creation 👤

1. Create a user for yourself on the server after connecting with SSH. To do so, run:

   ```
   sudo useradd USERNAME
   ```

   Replace `USERNAME` with your name.

2. Now, set a password for the new user:

   ```
   sudo passwd USERNAME
   ```

3. For the new user to be able to use `sudo`, it has to be added to the `wheel` group:

   ```
   sudo usermod -aG wheel USERNAME
   ```

   `-aG` stands for *append to group(s)*.

   (On Debian based distros, the user should be added to the `sudo` group instead of `wheel`.)

4. Now, change your user to the new user:

   ```
   su --login USERNAME
   ```

   You will be asked for the password. After a successful authentication, you will see that the username changed in the prompt.

5. Run the following command for verification:

   ```
   whoami
   ```

   It should not output "admin"!

   Yes, the command is called `whoami`. Linux can be philosophical sometimes 🤔

6. Now, verify that you can run `sudo` as the new user:

   ```
   sudo whoami
   ```

   You should see "root" as output because `sudo` runs a command temporarily as the `root` user.

7. `cd` to the home directory of the new user.

8. Make sure that you are in the home directory of the new user! Run `pwd` to verify that you are **NOT** in `/home/admin` ⚠️ **PLEASE, DON'T TOUCH `/home/admin/.ssh`** ⚠️ Now, create the directory `~/.ssh` in the home directory of the new user. Change the permissions of `~/.ssh` such that only the user has read, write and execution permissions. *group* and *others* should have no permissions for `~/.ssh`!

9. Create the file `authorized_keys` inside `~/.ssh`. Only the user should have read and write permissions for the file. *group* and *others* should have no permissions for it!

10. Copy the content of your public key file (with the extension `.pub`) to this file. It should be one line! Then save the file.

11. Logout from the server to get back to the system in the **browser terminal**. Go to `~/.ssh/config` that you edited at the beginning of this task. Change the user for the host `linux-lab` from `admin` to `USERNAME` where `USERNAME` is the name of the new user that you created on the server.

12. Try to connect using the host name again. If you did everything right, you should be connected and be the user that you did create. Run `whoami` to verify that the output is not "admin".

Task: File transfer ⬆️ ⬇️

In the system of the browser terminal, use `rsync` to upload some files and directories that you created during the course to the server `linux-lab` ⬆️

Now, login to the server with SSH to verify that the files and directories were uploaded correctly.

While on the server, create a file with some text in it. Remember its path!

Now, logout from the server and use `rsync` to download that file to your system ⬇️

Task: Compilation in containers 📦

📍 : This task should be done on the server using the user that you created (**not** admin).

We want to practice scripting and dealing with containers. Therefore, we will compile something in a container!

We want to compile the program `tmate`:

1. Start Zellij on the system of the browser terminal.
2. Login the user that you created on the server (**not** admin).
3. Start an Ubuntu container with `podman run -it --rm --name tmate-compiler ubuntu:latest bash`.

4. Run `apt update` to be able to install packages with the `apt` package manager in the next steps.
5. Go to the website of `tmate` and find out how to compile from **source** (there are instructions for compiling on Ubuntu).
6. Install the packages that are required for the compilation with `apt install`. These packages are listed on the website where the compilation instructions are.
7. Follow the actual compilation instructions on the website. The compilation might take some minutes.
8. After compilation, you will find the program file `tmate` in the directory of the git repository.
9. Don't exit the container yet, otherwise you will lose what you have done in it! Now, open a new Zellij pane, login to the same user on the server and copy the binary `tmate` from the container to the directory `bin` in your home directory. Use the command `podman cp CONTAINERNAME:SRC_PATH DESTINATION_PATH`.
10. Verify that the binary `tmate` was copied to `DESTINATION_PATH` and then exit the container in the first Zellij pane.

Now, write a script called `compile_tmate.sh` that automates what you have done in the container to compile `tmate`. Just copy all the commands that you used inside the container to a script.

Add `mv PATH_TO_THE_TMATE_PROGRAM_IN_THE_CONTAINER /volumes/bin` to the end of the script to move the binary to the directory `/volumes/bin` after compilation.

Create a directory called `scripts` and put the script in it.

Now, write a second script in the **parent directory** of the directory `scripts`. The second script should automate creating the container that runs the first script (`compile_tmate.sh`).

Do the following in the second script:

1. Check if `scripts/compile_tmate.sh` does NOT exist. In that case, print an error message and exit with the code 1.

2. Make sure that `scripts/compile_tmate.sh` is executable for the user.

3. Create a directory called `bin` (next to the directory `scripts`) if it doesn't already exist.

4. Use the following snippet:

```
podman run -it --rm \
    --name tmate-compiler \
    --volume ./scripts:/volumes/scripts:Z,ro \
    --volume ./bin:/volumes/bin:Z \
    docker.io/library/ubuntu:latest \
    /volumes/scripts/compile_tmate.sh
```

It creates a container that runs the script `compile_tmate.sh` and is removed afterwards (because of `--rm`).

The `scripts` directory is mounted as a volume to be able to give the container access to the script `compile_tmate.sh`. It is mounted as *read only* (`ro`) because it will not be modified.

The `bin` directory is mounted to be able to transfer the binary into it before the container exits.

After running the second script, you should see the container compiling and then exiting. At the end, you should find the binary `tmate` in the `bin` directory.

Now, that you have the program `tmate`, find out what it does! Try it with another participant 😀

Hints

- On Debian based distributions like Ubuntu, the package manager is `apt`. Before that you can install any packages with `apt`, you have to run `apt update`. This does not run system updates like `dnf upgrade`. `apt update` does only synchronize repositories which is needed before any installation.
- You can use the following snippet to test if a file exists in bash:

  ```
  if [ -f FILE_PATH ]
  then
    …
  fi
  ```

  Replace `…` with your code.

  For more information on the option `-f` and other useful options for bash conditions, read the man page of the program `test` inside of `bash`: `man test`.

To test if a file does NOT exist, replace `-f` with `! -f`.

- You can exit a Bash script returning an exit code using `exit`:

```
exit 1
```

Task: Static website 🌐

📍: In this task, you should connect as the user `admin` to the server. **Don't do this task as the user that you created on the server!** ⚠️ Just run `ssh admin@linux-lab`

📍: Starting with this task: Asking you to replace `N` means to enter the number that you are using in the URL `ttydN.mo8it.com`.

In this task, you will host a static website. A static website is just a set of HTML, CSS (and optionally JavaScript) files (no backend).

To host the website, we need a web server. In this task, we will use the Nginx web server.

Create the directory `~/nginxN` after replacing `N`. Create two directories inside it: `website` and `config`.

Place these two files:

1. `~/nginxN/config/nginx.conf` (replace `N`):

```
server {
    root /volumes/website;
    index index.html;

    location / {
    }
}
```

2. `~/nginxN/website/index.html` (replace `N`):

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Demo</title>
  </head>
  <body>
    <h1>Hello world!</h1>
  </body>
</html>
```

Create an Nginx container with the following options:

- Name: `nginxN`. Replace `N`!
- Network: `traefik`.
- Volumes:
  - `~/nginxN/website:/volumes/website` with labels `Z,ro`.
  - `~/nginxN/config:/etc/nginx/conf.d` with labels `Z,ro`.
- Image: `docker.io/library/nginx:latest`

Create the systemd service file for the container above.

Move the systemd service file to `~/.config/systemd/user`.

Enable and start the container as a user services with `systemctl --user enable --now container-nginxN`. Replace `N`!

Visit `https://nginxN.mo8it.xyz` in your browser to see if everything did work! Replace `N`!

Now, you can edit `index.html` and add your own HTML content.

You can also add more files to the directory `website`. If you add a file `test.html` for example, then you should see it under the link `https://nginxN.mo8it.xyz/test`.

Task: Nextcloud ☁️

📍: In this task, you should connect as the user `admin` to the server. **Don't do this task as the user that you created on the server!** ⚠️ Just run `ssh admin@linux-lab`

In this task, you will deploy your own cloud on the server: Nextcloud!

To do so, we will install Nextcloud as a container using `podman`.

You can find more information about the Nextcloud container here.

Create the directory `~/nextcloudN` (replace `N`).

Create a directory called `~nextcloudN-db` (replace `N`) for the database container.

Create a container for the database with the following options:

- Container name: `nextcloudN-db`. Replace `N`!
- Network: `traefik`
- Volume: Mount the directory `nextcloudN-db` (replace `N`) that you created into `/var/lib/postgresql/data` in the container. Use the label `Z`!
- The following environment variables:
  - `POSTGRES_DB=nextcloud`
  - `POSTGRES_USER=nextcloud`
  - `POSTGRES_PASSWORD=DB_PASSWORD`. Replace `DB_PASSWORD` with a good password!
- Image: `docker.io/library/postgres:16`

Create the actual Nextcloud container with the following options:

- Container name: `nextcloudN`. replace `N`!
- Network: `traefik`
- Volume: Mount the directory `nextcloudN` that you created into `/var/www/html` in the container. Use the label `Z`!
- The same environment variables as for the other container! Use the same `DB_PASSWORD`. Add one more environment variable:
  - `POSTGRES_HOST=nextcloudN-db`. Replace `N`!
- Image: `docker.io/library/nextcloud:27-apache`

Create the systemd files for both containers. Move the systemd files to `~/.config/systemd/user`.

Enable and start both containers as a user services with `systemctl --user enable --now container-nextcloudN-db` and `systemctl --user enable --now container-nextcloudN`. Replace `N`!

Visit `https://nextcloudN.mo8it.xyz` to see if everything did work! Replace `N`!